DEVSECOPS LAB BOOK · ZERO TO HERO

Phase 2: Immutable Infrastructure

COMPLETE ✅

Packer builds the image. OpenTofu provisions the VMs. Ansible configures the services. The entire lab now deploys from scratch in 30 minutes.

Recap & Handoff Deck — Chapters 07–09

WHAT THIS RECAP COVERS

From Manual Clicks to One Command

Chapter Recap

What Packer, OpenTofu, and Ansible each deliver

Manual vs Automated

8+ hours of clicking vs. 30 minutes of code

The Automation Stack

How image → infrastructure → config chain together

Real Scenarios

Deploying from scratch, updating a service, adding a VM

Security Gains

What automation fixes that manual work couldn't

What's Next

Phase 3, Phase 4, or a security deep-dive

🎉 MILESTONE

Full Automation Achieved

CHAPTER 07

Packer

Build golden images from code

CHAPTER 08

OpenTofu

Provision 12 VMs with one command

CHAPTER 09

Ansible

Configure all services automatically

TOTAL INVESTMENT

3 chapters · 100+ pages · 40,000+ words

RESULT

Entire lab deploys in **30 minutes** (vs. 8+ hours manual)

WHAT PHASE 2 ACHIEVES

From Manual to Automated

Task	Chapters 01–06 (Manual)	Phase 2 (Automated)	Time Saved
Build base image	45 min (clicks + hardening)	5 min (packer build)	**40 min**
Create 12 VMs	90 min (virt-install + wait)	5 min (tofu apply)	**85 min**
Configure services	180 min (SSH to each, copy configs)	20 min (ansible-playbook)	**160 min**

TOTAL

315 min (5+ hrs) → 30 min

SAVED

285 min (4.75 hrs)

Every minute saved compounds — this isn't a one-time build, it's every rebuild, every new environment, every disaster recovery.

THE AUTOMATION CHAIN

Packer → OpenTofu → Ansible

Packer

Image Building

- Build base OS + hardening
- Run provisioning scripts
- Output: QCOW2 image

OpenTofu

Infrastructure as Code

- Define all 12 VMs in HCL
- Provision network + VMs
- Clone image to each VM
- Output: Running VMs

Ansible

Configuration Management

- SSH to each VM
- Install packages + services
- Deploy configuration files
- Output: Fully configured lab

CHAPTER RECAP

Ch. 07 — Packer: Golden Image Creation

45 pages

DELIVERABLES

- Packer template in HCL (version-controlled)
- Provisioning scripts (packages, hardening, cleanup)
- Automated image building — no manual steps
- QCOW2 output (KVM-compatible)
- Validation script

KEY CONCEPTS

- Builders (QEMU for KVM)
- Provisioners (shell scripts)
- Post-processors (manifest generation)
- Variable definitions (configurable build)

BENEFITS

✓ Identical images every time ✓ Changes version-controlled ✓ Builds are auditable ✓ Rebuild fast after security patches

CHAPTER RECAP

Ch. 08 — OpenTofu: Infrastructure as Code

45 pages

DELIVERABLES

- OpenTofu provider configuration (libvirt for KVM)
- Network definition (10.0.2.0/24 isolated network)
- All 12 VM definitions (IP, vCPU, RAM, disk)
- State management (terraform.tfstate)
- Variables + outputs (IPs, SSH commands)

KEY CONCEPTS

- Providers (connect to KVM)
- Resources (VMs, networks, disks)
- Variables (configurable input)
- State (track infrastructure)
- Modules (reusable components)

BENEFITS

✓ Entire lab defined in code ✓ Changes reviewable (tofu plan) ✓ Disaster recovery < 1 hour ✓ Audit trail via Git

CHAPTER RECAP

Ch. 09 — Ansible: Configuration Management

45 pages

DELIVERABLES

- Ansible inventory (12 VMs organized by role)
- Common role (base config all VMs share)
- Service-specific roles (BIND9, LLDAP, HAProxy, etc.)
- Jinja2 templates (dynamic config files)
- Main playbook + handlers (restart on change)

KEY CONCEPTS

- Inventory (which VMs to manage)
- Playbooks (list of plays/tasks)
- Roles (reusable configuration units)
- Handlers (event-driven actions)
- Jinja2 templates (dynamic configs)

BENEFITS

✓ Idempotent — safe to re-run ✓ Agentless (SSH only) ✓ Human-readable YAML ✓ Selective execution by role/host

Scenario: Deploy the Lab From Scratch

```
# 1. Build the image (5 min)
$ cd ~/lab/packer/ubuntu-22.04
$ packer build .
→ Image built: ubuntu-22.04-packer.qcow2

# 2. Provision VMs (10 min)
$ cd ~/lab/opentofu/devsecops-lab
$ tofu init && tofu plan && tofu apply
→ Created 12 VMs, network, all resources

# 3. Configure services (20 min)
$ cd ~/lab/ansible/devsecops-lab
$ ansible-playbook playbooks/site.yml
→ All VMs updated, services installed and running
```

Total time: ~35 minutes Manual approach: 5–8 hours

Scenario: Update a Service, Add a VM

Update BIND9 configuration

```
$ vim roles/bind9/templates/named.conf.options.j2
$ git commit -am 'Update DNS config'
$ ansible-playbook -t bind9 playbooks/site.yml
→ BIND9 updated & reloaded — no VM rebuild needed
```

Add a new VM

```
$ vim variables.tf   # add new VM definition
$ tofu plan     → Plan: 1 to add
$ tofu apply    → New VM created
$ ansible-playbook -l new-vm-group playbooks/site.yml
→ New VM fully configured
```



The pattern that repeats

Edit code → review the plan → apply the change → done. Whether it's a config tweak or a brand-new VM, the workflow is identical: no memorized steps, no SSH-and-hope. The code is the single source of truth, and every change is a Git commit.

REPOSITORY LAYOUT

How the Phase 2 Files Are Organized

```
~/lab/
├── packer/ubuntu-22.04/
│   ├── ubuntu-22.04.pkr.hcl   # Main template
│   ├── variables.pkr.hcl
│   ├── scripts/               # install, harden, cleanup
│   └── output/                # Built image
│
├── opentofu/devsecops-lab/
│   ├── providers.tf  variables.tf  network.tf  vms.tf
│   ├── outputs.tf  terraform.tfvars  terraform.tfstate
│   └── environments/
│
└── ansible/devsecops-lab/
    ├── ansible.cfg  inventory.ini
    ├── playbooks/   site.yml, dns.yml, identity.yml, edge.yml
    ├── roles/       common, bind9, lldap, authentik,
    │                haproxy, nginx, oauth2-proxy
    └── templates/ + files/
```

KEY LEARNINGS

Four Ideas Worth Remembering



Reproducibility is Powerful

Same deployment every time. No "works on my machine." Test changes safely with plan-before-apply. Recover from disasters in 30 minutes.



Infrastructure Becomes Auditable

Every change tracked — who, when, what. See diffs before applying. Roll back to any previous version. Compliance audits become a `git log`.



Scaling Becomes Trivial

Change a variable file, run `tofu apply -var-file=prod.tfvars`. An entire new environment exists — zero manual work per environment.



Configuration Drift Disappears

No more "VM A has nginx 1.18, VM B has 1.20." Every Ansible run enforces desired state. Configuration is the single source of truth.

SECURITY IMPROVEMENTS

What Automation Fixed

PHASE 1 — MANUAL

- ✗ Manual steps prone to error
- ✗ Hard to audit what was configured
- ✗ Inconsistent configurations across VMs
- ✗ Difficult to apply security patches

PHASE 2 — AUTOMATED

- ✓ Code review before deployment (pull requests)
- ✓ Full audit trail (Git history)
- ✓ Identical hardening on all VMs
- ✓ Easy security-patch updates — edit + re-run
- ✓ Test changes in dev before prod
- ✓ Roll back quickly if something breaks

CAPABILITIES UNLOCKED

What's Now Possible

Deploy instantly

30 minutes to a full lab

Update safely

Plan, review, apply with confidence

Recover quickly

Disaster = 1 hour to rebuild

Scale horizontally

Multiple environments, same code

Audit everything

Git history of every change

Test changes

Run playbooks on dev before prod

Teach others

Code is the documentation

Collaborate

Engineers review & modify together

YOUR DECISION

Next Steps

A

Continue to Phase 3

Vault (secrets management) + Teleport (zero-trust access)

Recommended · est. 3–4 weeks

B

Jump to Phase 4

GitLab, SonarQube, Harbor — full CI/CD supply chain security

est. 2–3 weeks

C

Add Security Deep-Dives

Production hardening for BIND9, Ansible, and OpenTofu

est. 1–2 weeks

PHASE 2 · IMMUTABLE INFRASTRUCTURE

Infrastructure is Code.

The lab can now be deployed, destroyed, and redeployed infinitely and identically. Packer builds it, OpenTofu provisions it, Ansible configures it.

8+ hrs → 30 min	3	135+
Automation achieved	Chapters completed	Pages of content

Ready to proceed? Say: “Proceed with Phase 3: Vault & Teleport” — or choose your own path.